

M.S. Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Dept. of AI & ML/ Dept. of AI & DS

Course Name: Introduction to Artificial Intelligence(AI)
Course Code: AI35/AD35
Credits: 3:0:0
2023 Batch

UNIT -2 Outline

Logical Agents (Chapter 7)

- Knowledge-based agents (7.1)
- Wumpus world(7.2)

First Order Logic

- Representation revisited
- Syntax and semantics of first-order logic
- Knowledge engineering in first-order logic

Inference in First Order Logic

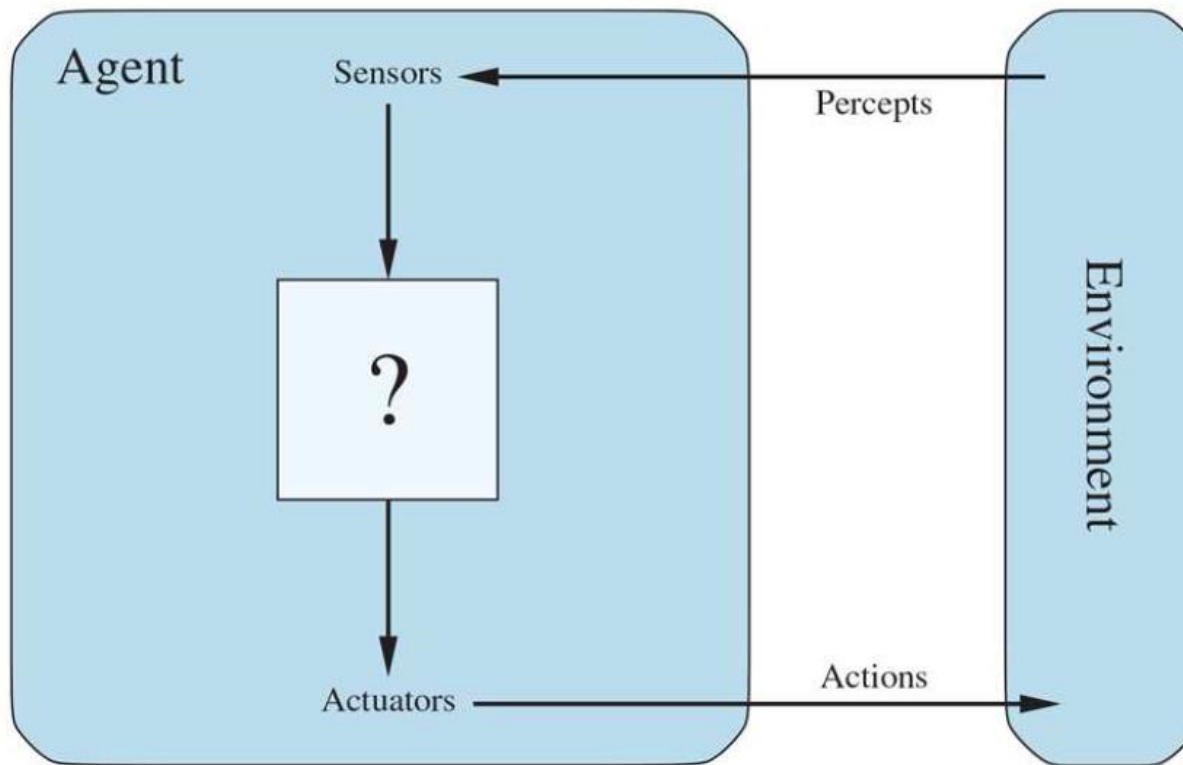
- Propositional vs. First-order inference
- Forward chaining
- Backward chaining

Introduction

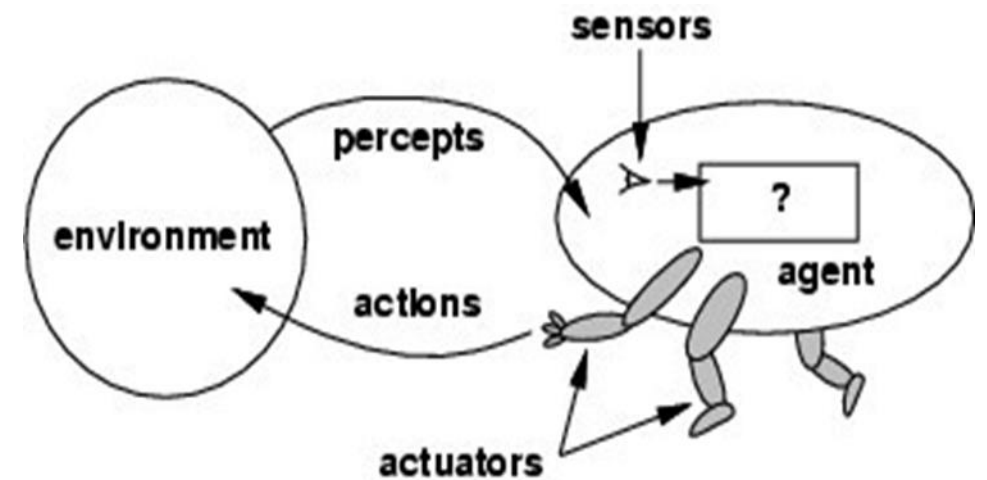
- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.
- Knowledge - facts, skills, education, experience
- Intelligence – ability to use that knowledge
- Reasoning – Processing of Knowledge

Introduction

Figure 2.1



Agents interact with environments through sensors and actuators.



Logical Agents

- The **problem-solving agents** know things, but only in a very limited, inflexible sense.
- They know what actions are available and what the result of performing a specific action from a specific state will be, **but they don't know general facts.**
- Example: A route finding agent

Knowledge Based Agent

“Humidity is high”

“Temperature is high”

“One feels comfortable”.

- Knowledge-based agent has its knowledge base, or **KB**.
- A knowledge base is a set of **sentences**.
- Each sentence is expressed in a language called a **knowledge representation language** and represents some assertion/facts about the world.
- There must be a way to **add new sentences** to the knowledge base and a way to **query** what is known.
- Both operations may involve **inference**— that is, deriving new sentences from old.

Knowledge Based Agent

- The knowledge is typically **represented in the form of rules or facts**, which can be used to draw conclusions or make decisions.
- **Benefits**: It can help to **automate decision-making processes**.

Example:

- A knowledge-based system could be used to **diagnose** a medical condition, by reasoning over a set of rules that describe the **symptoms** and possible causes of the condition.
- **Applications** : Medical diagnosis, Expert systems, and Decision support systems.

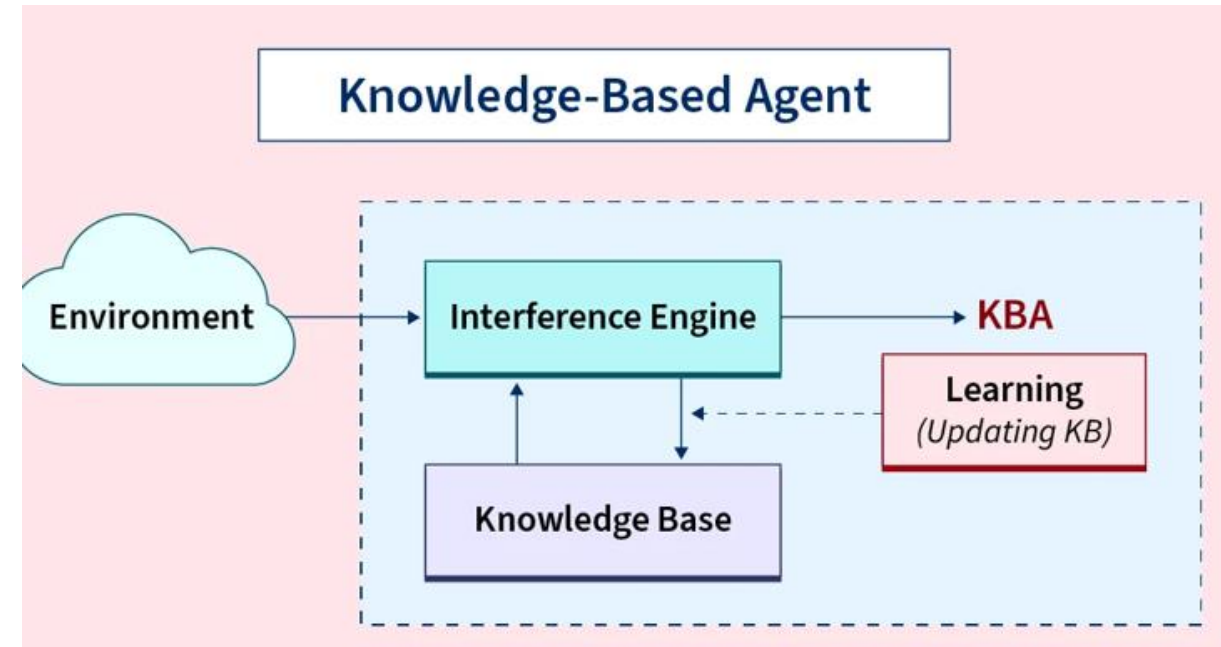
Knowledge Based Agent

Knowledge Base:

- It is a **collection of sentences** (here 'sentence' is a technical term and it is not identical to sentence in English).
- These sentences are expressed in a language which is called a **knowledge representation language**.
- The Knowledge-base of KBA stores **fact** about the world.

Inference system

- Inference means **deriving new sentences from old**.
- Inference system allows us to **add a new sentence to the knowledge base**.



Knowledge based Agent

Central component of Knowledge based Agent is a Knowledge-base(KB)

KB = set of sentences in a formal language

Two generic functions are

1. **TELL** – add new sentence to KB (TELL IT WHAT IT NEEDS TO KNOW)
2. **ASK** – Query what is know from KB (ASK WHAT TO DO NEXT)

For example:

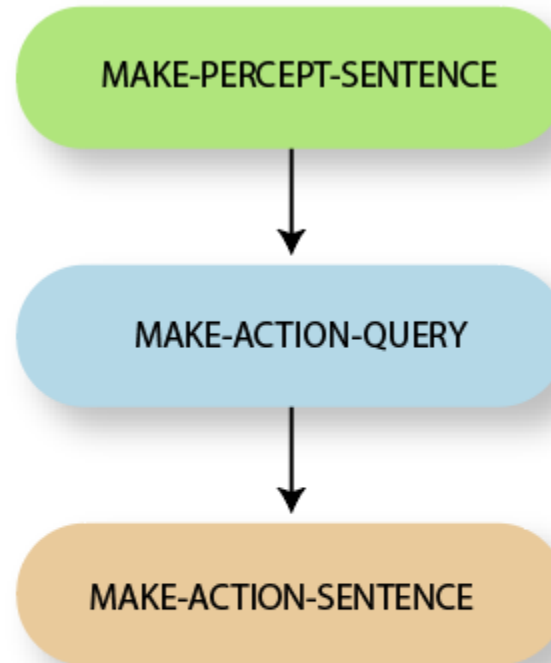
- ▶ TELL: Father of John is Bob
- ▶ TELL: Jane is John's sister
- ▶ TELL: John's Father is the same as John's sister's father
- ▶ ASK: Who's Jane's Father?

Tell facts
Ask for inference

Knowledge based Agent

Agent maintains a KB in 3 steps:

1. First it TELLS the KB what it perceives
2. Second ASK the KB what action it should perform
3. Third agent program TELLS the knowledge base which action was chosen, and returns the action so that it can be executed.



MAKE-PERCEPT-SENTENCE Generates a sentence, what the agent perceived at the given time.

MAKE-ACTION-QUERY Generates a sentence to ask which action should be done at the current time.

MAKE-ACTION-SENTENCE generates a sentence which asserts that the chosen action was executed.

Functions hiding details of Knowledge Representation Language

Knowledge based Agent

```
function KB-AGENT(percept) returns an action
  static: KB, a knowledge base
         t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

1. It TELLS the KB what it perceives
2. It asks the KB what action it should perform
3. The agent program TELLS the knowledge base which action was chosen, and returns the action so that it can be executed.

- MAKE-PERCEPT-SENTENCE generates a sentence, what the agent perceived at the given time.
- MAKE-ACTION-QUERY generates a sentence to ask which action should be done at the current time.
- MAKE-ACTION-SENTENCE generates a sentence which asserts that the chosen action was executed.

-
- There are mainly two approaches to build a knowledge-based agent:
 - **Declarative approach:** A knowledge-based agent can be built simply by TELLing it what it needs to know. Starting with an empty knowledge base, the agent designer can TELL sentences one by one until the agent knows how to operate in its environment. This is called the declarative approach to system building.
 - **Example : Constraints and rules**
 - **Procedural approach:** We directly encode desired behavior as a program code. Which means we just need to write a program that already encodes the desired behavior or agent.
 - **Example: Functions**
 - Successful agent often combines both declarative and procedural elements in its design, and that declarative knowledge can often be compiled into more efficient procedural code.

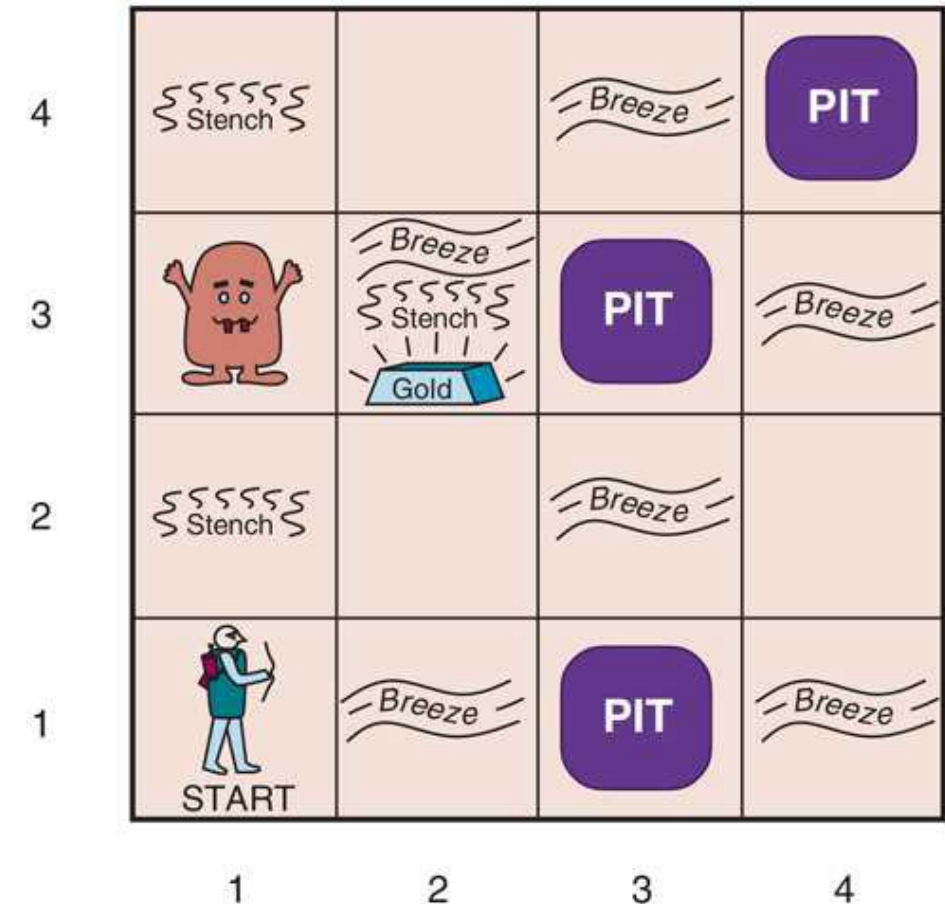
Wumpus World

Wumpus world – Environment to illustrate the worth of a knowledge based agent and to represent knowledge

It was inspired by a video game Hunt the Wumpus by Gregory Yob in 1973

A Wumpus World

- The wumpus world is a **cave** consisting of rooms connected by passageways.
- Terrible wumpus, is a **beast that eats** anyone who enters its room.
- The wumpus can be shot by an agent, but the agent has only **one arrow**.
- Some rooms contain **bottomless pits** that will trap anyone who wanders into these rooms.
- **Pits** are surrounded by Breeze and **Wumpus** is surrounded by Stench.
- **Goal** – Agent should pick the Gold present in a Particular room



Wumpus World PEAS description

Performance measure

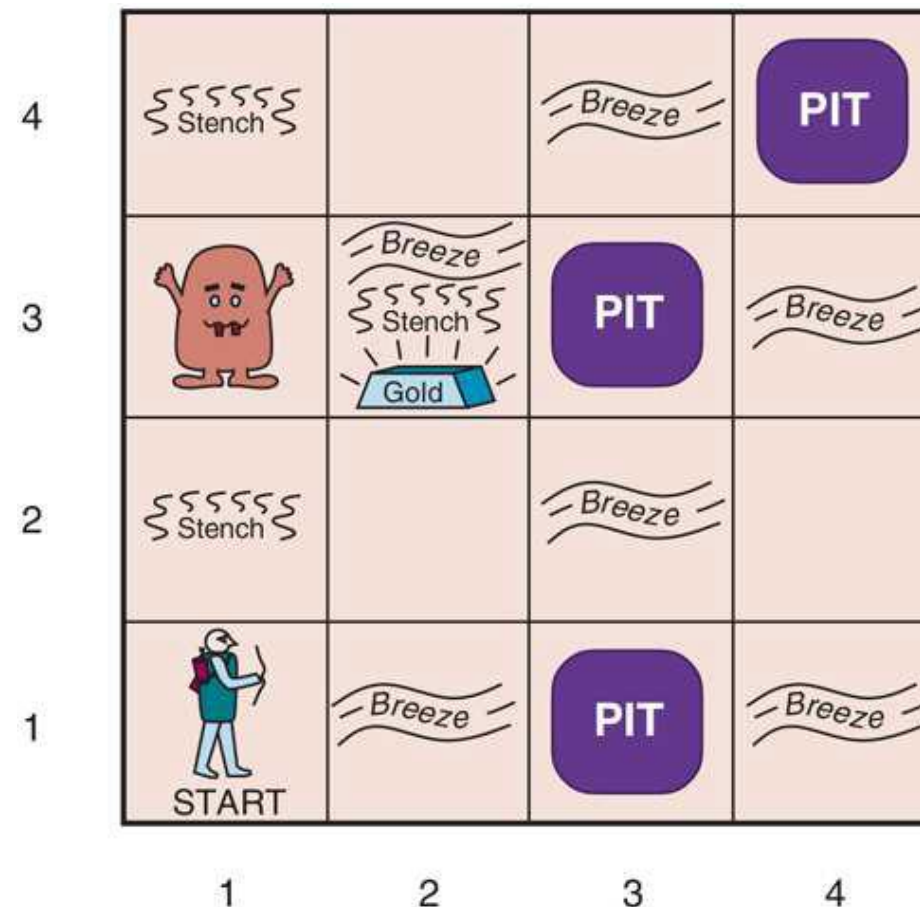
- Gold = +1000, Death = -1000
- -1 per step, -10 for using the arrow
- Game ends either when the agent dies or when the agent climbs out of the cave.

Environment: 4 x 4 grid of rooms

- [1,1] start of an Agent
- Locations of the gold and the wumpus are chosen randomly

Sensors: Stench, Breeze, Glitter, Bump(Hit wall), Scream (shot Wumpus)

Actuators: Left turn, Right turn, Forward, Grab, Release, Shoot

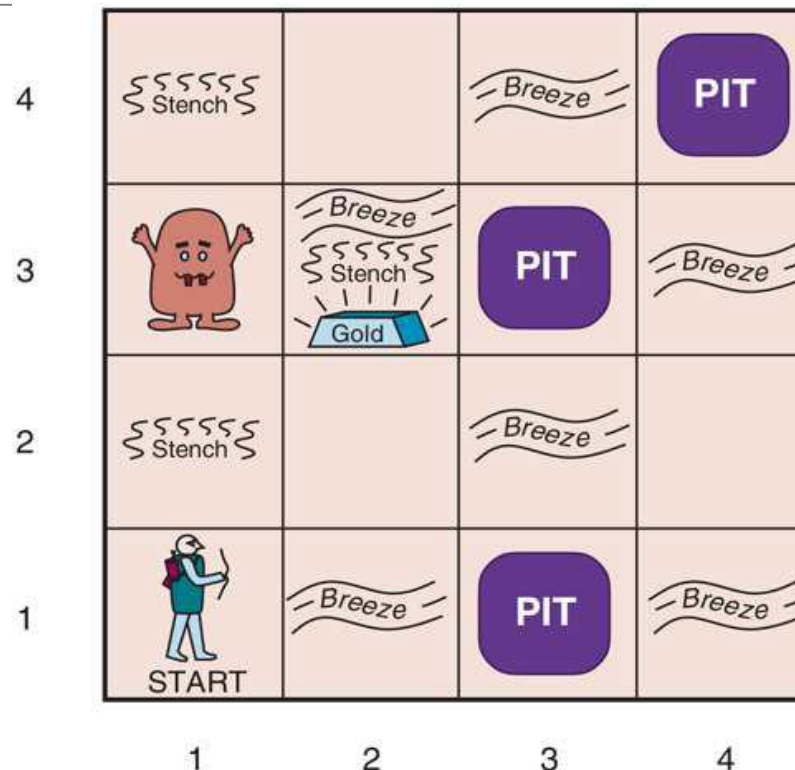


Wumpus World

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1	2,1	3,1	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

(a)



Percepts given to the agent

1. Stench
2. Breeze
3. Glitter
4. Bumb (ran into a wall),
5. Scream (wumpus has been hit by arrow)

Figure 7.3: The first step taken by the agent in the wumpus world. (a) The initial situation, after percept **[None,None,None,None,None]**. **[Stench,Breeze,Glitter,Bumb,Scream]**

Wumpus World

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1	2,1	3,1	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

(a)

Step 1: The first step taken by the agent in the wumpus world. The initial situation, after percept **[None, None, None, None, None]**.

Step 2: After moving to [2,1] and perceiving **[None, Breeze, None, None, None]**. Wumpus realizes there may be pit in adjacent cells so moves back to [1,1].

Step 3: After moving to the agent perceives a stench in [1,2], resulting in **[Stench, None, None, None, None]** the state of knowledge that there must be a wumpus nearby. But the wumpus cannot be in [1,1], by the rules of the game, and it cannot be in [2,2] (or the agent would have detected a stench when it was in [2,1]).

Step 4: After moving to [2,2] **[None, None, None, None, None]**. Therefore, the agent can infer that the wumpus is in [1,3]. The notation W! indicates this inference.

Moreover, the lack of a breeze in [1,2] implies that there is no pit in [2,2]. Yet the agent has already inferred that there must be a pit in either [2,2] or [3,1], so this means it must be in [3,1].

Step 5: After moving to [2,3], and perceiving **[Stench, Breeze, Glitter, None, None]**.

First Order logic

- Propositional Logic Basics
- Representation revisited
- Syntax and semantics of first-order logic
- Knowledge engineering in first-order logic

Propositional Logic

- A proposition is a declarative statement which is either true or false.
- It is a technique of knowledge representation in logical and mathematical form.

Example:

- a) It is Sunday.
- b) The Sun rises from West (False proposition)
- c) $3+3=7$ (False proposition)
- d) 5 is a prime number.

Propositional Logic

- Propositional logic is also called **Boolean logic** as it works on 0 and 1.
- In propositional logic, **symbolic variables** are used to represent the logic, such A, B, C, P, Q, R, etc.
- **Ex: A:It is Sunday**
- Propositions can be either true or false, but it **cannot be both**.
- **Connectives** can be used as a **logical operator** which connects two sentences.
- The propositions and connectives are the **basic elements** of the propositional logic.

Propositional Logic

- The Atomic sentences consist of a single proposition symbol.
- Complex sentences are constructed from simpler sentences, using parentheses and logical connectives.

Type	Connective	Symbol	Notation	Read as
Conjunction	And	$\wedge$	$P \wedge Q$	P and Q
Disjunction	Or	$\vee$	$P \vee Q$	P or Q
Conditional	If then	$\rightarrow$	$P \rightarrow Q$	P implies Q i.e If P then Q
Bi-conditional	If and only if	$\leftrightarrow$	$P \leftrightarrow Q$	P double implies Q i.e. P if and only if Q
Negation	Not or No	$\sim$ or $\neg$	$\sim P$	Negation P (or) Not P

Propositional Logic

All sentences are constructed from atomic sentences and the five connectives are expressed with truth tables

- $\neg P$ is true iff P is false in m .
- $P \wedge Q$ is true iff both P and Q are true in m .
- $P \vee Q$ is true iff either P or Q is true in m .
- $P \Rightarrow Q$ is true unless P is true and Q is false in m .
- $P \Leftrightarrow Q$ is true iff P and Q are both true or both false in m .

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Propositional Logic

$P_{x,y}$ is true if there is a pit in $[x,y]$.

$W_{x,y}$ is true if there is a wumpus in $[x,y]$, dead or alive.

$B_{x,y}$ is true if there is a breeze in $[x,y]$.

$S_{x,y}$ is true if there is a stench in $[x,y]$.

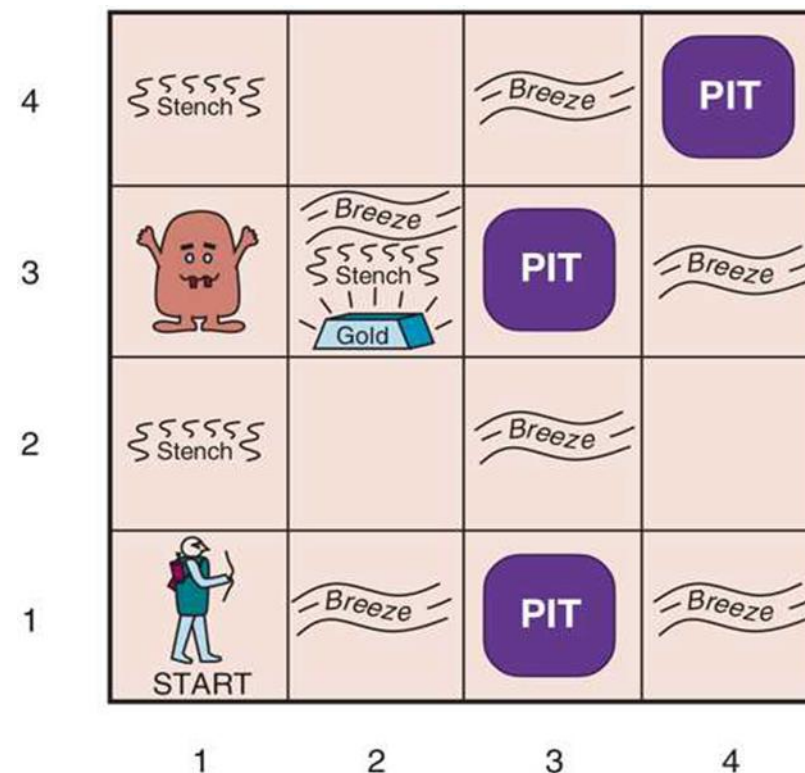
- There is no pit in $[1,1]$:

$$R_1: \neg P_{1,1}.$$

- A square is breezy if and only if there is a pit in a neighboring square. This has to be stated for each square; for now, we include just the relevant squares:

$$R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}).$$

$$R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}).$$



Models for first-order logic

- **Atomic sentences:**

- Atomic sentences are the **most basic sentences** of first-order logic.
- These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- We can represent atomic sentences as **Predicate (term1, term2,, term n)**.
- **Example: Richard and John are Brothers**
- **Brother (Richard , John)**
- *An atomic sentence is true in a given model if the relation referred to by the predicate symbol holds among the objects referred to by the arguments.*

- **Complex Sentences:**

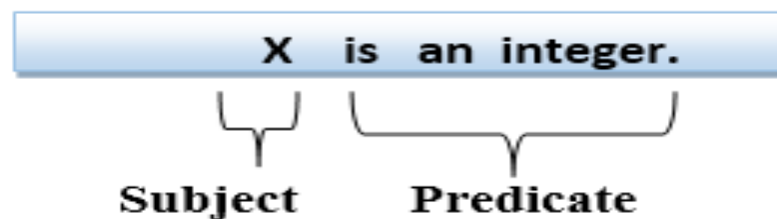
- Complex sentences are made by combining atomic sentences using connectives.
- Example : **King(Richard) $\vee$ King(John)**

Representation Revisited

First-order logic statements can be divided into two parts:

Subject: Subject is the main part of the statement.

Predicate: A predicate can be defined as a relation, which binds two atoms together in a statement



First Order Logic

- FOL is a collection of objects, relations and functions among them to represent knowledge.
- **Objects** (also known as subjects) are the entities or things we refer to in a domain. They are the primary items that make up the "world" we're describing in first-order logic.
- These could be concrete items (like a person or city) or abstract concepts.
- Examples of Objects: People: Alice, John, Mary
- Places: Paris, New York, India
- Things: Car, Book, Tree
- Numbers: 5, 42
- In FOL, objects are represented by constants or variables, typically using lowercase letters like a, b, x, or full names like Alice.

- **Relations** (also known as predicates) describe how objects are related to each other or describe properties of objects.
- They are statements that can be true or false based on the objects involved.
- **Unary Relations:** These involve a single object and state a property of that object.
 - Example: IsHuman(John) - "John is human."
- **Binary Relations:** These involve two objects and describe a relationship between them.
 - Example: Parent(John, Mary) - "John is a parent of Mary."
- **N-ary Relations:** These involve more than two objects and describe complex relationships.
 - Example: Between(Paris, Rome, Berlin) - "Paris is between Rome and Berlin."
- **Relations are usually written as predicates that take objects as arguments and return a boolean (true or false) value.**

- **Functions** in first-order logic are similar to mathematical functions: they take one or more objects as input and return exactly one object as output.
- Unlike relations, which return true or false, functions return a specific object or value.
- Function Notation: Functions are often represented by **lowercase letters** and typically appear within a predicate.
- **Example:**
 - MotherOf(John): This returns the object that is John's mother.
 - CapitalOf(France): This might return Paris.
 - AgeOf(Alice): This might return 30, representing Alice's age.
 - FatherOf(Mary): This might return John, if John is Mary's father.
- **Functions allow us to retrieve specific pieces of information from objects in a structured way.**

Real World Scenario : Healthcare Assistant

Let's assume we are building a healthcare assistant to manage patient information, doctor schedules, and treatment recommendations.

1. **Objects** are entities involved in the system. In this scenario, objects could include people, places, and items:

Doctors: DrSmith, DrJones

Patients: Alice, Bob

Rooms: Room101, Room202

Diseases: Flu, Diabetes

Treatments: Insulin, Rest, Antibiotics

Real World Scenario : Healthcare Assistant

2. Relations describe how these objects interact or are associated with each other in the healthcare setting.

Examples of Relations:

IsDoctor(DrSmith): Dr. Smith is a doctor.

IsPatient(Alice): Alice is a patient.

Treats(DrSmith, Alice): Dr. Smith treats Alice.

AssignedRoom(Alice, Room101): Alice is assigned to Room 101.

HasDisease(Alice, Flu): Alice has the flu.

Prescribes(DrSmith, Alice, Rest): Dr. Smith prescribes rest for Alice.

ScheduledAppointment(Alice, DrSmith, Monday): Alice has an appointment with Dr. Smith on Monday.

These relations allow us to express statements about the hospital environment in logical terms. Each relation is either true or false depending on the current state of the hospital system.

Real World Scenario : Healthcare Assistant

3. **Functions** produce specific values or objects based on other objects. They help retrieve or compute information based on known relations.

Examples of Functions:

DoctorOf(Alice): This function returns DrSmith, indicating that Dr. Smith is Alice's doctor.

RoomOf(Alice): Returns Room101, which is Alice's assigned room.

TreatmentFor(Flu): Returns Rest, suggesting rest as the standard treatment for flu.

NextAppointment(Alice): Returns the day or time of Alice's next appointment with her doctor.

Functions help the system calculate or identify specific, unique details about each entity involved in the scenario.

Real World Scenario : Healthcare Assistant

Example of Logical Statements

Using these objects, relations, and functions, the healthcare assistant can derive logical statements to represent knowledge in the system:

IsDoctor(DrSmith): Dr. Smith is a doctor (a unary relation describing a property).

Treats(DrSmith, Alice): Dr. Smith is treating Alice (a binary relation).

ScheduledAppointment(Alice, DrSmith, Monday): Alice has an appointment with Dr. Smith on Monday (a ternary relation).

TreatmentFor(Flu) = Rest: The recommended treatment for flu is rest (a function returning an object).

RoomOf(Alice) = Room101: Alice is assigned to Room 101 (a function returning a room).

Syntax and Semantics of First-Order Logic

First order Logic (FOL) has two main parts:

Syntax

Semantics

Syntax: how the language facilitates concise representations

Semantics leads to sound reasoning procedures.

Write Statement in first-order logic

- **Ravi and Ajay are brothers: $\Rightarrow$ Brothers(Ravi, Ajay).**
- **Chinky is a cat: $\Rightarrow$ cat (Chinky).**

Beautiful(y) $\rightarrow$ “ y is beautiful”

Mother(x) $\rightarrow$ “ x is a Mother”

BrotherOf(x, y) $\rightarrow$ “ x is a brother of y ”

NextTo(x, y) $\rightarrow$ “ x is next to y ”

} Predicates

BrotherOf(x) $\rightarrow$ “the brother of x ”

NextTo(y) $\rightarrow$ “the thing next to y ”

SquareRoot(x) $\rightarrow$ “the square root of x ”

} Functions

Symbols and interpretations

Constant	1, 2, A, John, Mumbai, cat,...
Variables	x, y, z, a, b,....
Predicates	Brother, Father, >,....
Function	sqrt, LeftLegOf,
Connectives	$\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$
Equality	$=$
Quantifier	$\forall, \exists$

```

Sentence → AtomicSentence | ComplexSentence
AtomicSentence → Predicate | Predicate(Term,...) | Term = Term
ComplexSentence → ( Sentence ) | [ Sentence ]
                | ¬ Sentence
                | Sentence ∧ Sentence
                | Sentence ∨ Sentence
                | Sentence ⇒ Sentence
                | Sentence ⇔ Sentence
                | Quantifier Variable,... Sentence

Term → Function(Term,...)
      | Constant
      | Variable

Quantifier → ∀ | ∃
Constant → A | X1 | John | ...
Variable → a | x | s | ...
Predicate → True | False | After | Loves | Raining | ...
Function → Mother | LeftLeg | ...

OPERATOR PRECEDENCE : ¬, =, ∧, ∨, ⇒, ⇔
  
```

Figure 8.3 The syntax of first-order logic with equality, specified in Backus–Naur form

Write Statement in first-order logic

1. Marcus was a man

$\text{man}(\text{marcus})$

2. X is an integer

$\text{integer}(x)$

3. Caesar was a ruler

$\text{ruler}(\text{caesar})$

4. Bill is a Student

$\text{Student}(\text{Bill})$

5. Bill Takes either Analysis or Geometry

$\text{Takes}(\text{Bill}, \text{Analysis}) \vee \text{Takes}(\text{Bill}, \text{Geometry})$

6. Ram takes Biology if and only if Ram does not takes Maths

$\text{Takes}(\text{Ram}, \text{Biology}) \leftrightarrow \neg \text{Takes}(\text{Ram}, \text{Maths})$

Write Statement in first-order logic

7. Ravi and Rama are brothers

$\text{brothers}(\text{ravi}, \text{rama})$

8. Marcus tried to assassinate Caesar

$\text{try to assassinate}(\text{marcus}, \text{Caesar})$

9. if x is a philosopher, then x is a scholar"

$\text{Philosopher}(x) \rightarrow \text{scholar}(x)$

Write Statement in First-Order Logic(FOL) /Predicate Logic(PL)

1. "If it rains, then the ground is wet"

Let p: It rains

Let q: The ground is wet

Formal notation: $p \rightarrow q$

2. "The light is on if and only if the switch is up"

Let p: The light is on

Let q: The switch is up

Formal notation: $p \leftrightarrow q$

Write Statement in first-order logic

3. "If you study hard, you'll pass the exam. If you pass the exam, you'll graduate."

Let p: You study hard

Let q: You pass the exam

Let r: You'll graduate

Formal notation: $(p \rightarrow q) \wedge (q \rightarrow r)$

Can be simplified to: $p \rightarrow r$ (by transitivity)

4. "A triangle is equilateral if and only if all its angles are equal and all its sides are equal"

Let p: Triangle is equilateral

Let q: All angles are equal

Let r: All sides are equal

Formal notation: $p \leftrightarrow (q \wedge r)$

Write Statement in first-order logic

5. "If a number is prime, then it has exactly two factors"

Let p: The number is prime

Let q: The number has exactly two factors

Original implication: $p \rightarrow q$

6. "If a number doesn't have exactly two factors, then it's not prime"

$\neg q \rightarrow \neg p$

7. "A student passes the course if and only if they complete all assignments and if they complete all assignments, they must attend the final exam"

Let p: Student passes course

Let q: Student completes assignments

Let r: Student attends final exam

Formal notation: $(p \leftrightarrow q) \wedge (q \rightarrow r)$

Write Statement in first-order logic

8. "If it's sunny, then if it's warm, people will go to the beach"

Let p: It's sunny

Let q: It's warm

Let r: People go to the beach

Formal notation: $p \rightarrow (q \rightarrow r)$

9. "A circle has radius r if and only if its area is πr^2 and its circumference is $2\pi r$ "

Let p: Circle has radius r

Let q: Area is πr^2

Let r: Circumference is $2\pi r$

Formal notation: $p \leftrightarrow (q \wedge r)$

Write Statement in first-order logic

10. "A user can access the admin panel if and only if they're logged in and if they're logged in, they must have valid credentials"

Let p: User can access admin panel

Let q: User is logged in

Let r: User has valid credentials

Formal notation: $(p \leftrightarrow q) \wedge (q \rightarrow r)$

11. "A function is differentiable at a point if and only if it is continuous at that point and its derivative exists at that point"

Let p: Function is differentiable

Let q: Function is continuous

Let r: Derivative exists

Formal notation: $p \leftrightarrow (q \wedge r)$

Write Statement in first-order logic

12. "An integer is divisible by 6 if and only if it's divisible by 2 and 3"

Let p: Number is divisible by 6

Let q: Number is divisible by 2

Let r: Number is divisible by 3

Formal notation: $p \leftrightarrow (q \wedge r)$

13. "If you have a ticket or a pass, then you can enter the concert"

Let p: You have a ticket

Let q: You have a pass

Let r: You can enter the concert

Formal notation: $(p \vee q) \rightarrow r$

First Order logic

Quantifiers in First-order logic

A quantifier is a language element which generates quantification, and quantification **specifies the quantity of specimen** in the universe of discourse.

These are the symbols that permit to determine or **identify the range and scope** of the variable in the logical expression.

There are **two types of quantifier**:

- **Universal Quantifier(for all, everyone, everything)**
- **Existential quantifier(for some, at least one).**

First Order logic

Universal Quantifier

Universal quantifier is a symbol of logical representation, which specifies that the statement **within its range is true for everything** or every instance of a particular thing.

The Universal quantifier is represented by a **symbol $\forall$** , which resembles an inverted A.

The **main connective** for universal quantifier $\forall$ is implication $\rightarrow$.

Example : “All kings are persons” $\forall x \quad \text{King}(x) \Rightarrow \text{Person}(x)$

For all x, if x is a king, then x is a person.

- Ex: "All cats are mammals"

First Order logic

Existential Quantifier

Existential quantifiers express that the statement within its scope is true for at least one instance of something.

- It is denoted by the logical operator $\exists$, which resembles as inverted E.
- The **main connective** for existential quantifier $\exists$ is and $\wedge$.
- Example : King John has a crown on his head

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

$\exists x$ is pronounced “There exists an x such that . . .” or “For some x . . .”.

- Ex: "Some birds can fly"

Example: Write statement in FOL

1. All man drink coffee

$\forall x (\text{man}(x) \rightarrow \text{drink}(x, \text{coffee}))$

2. Every man respects his parent

$\forall x (\text{man}(x) \rightarrow \text{respects}(x, \text{parent}))$

3. Some boys play cricket

$\exists x (\text{boys}(x) \wedge \text{play}(x, \text{cricket}))$

Example : Write statement in FOL

4. All students like both maths and science

5. Not all students like both maths and science

7. “All boys like cricket”

8. “Some boys like cricket”

Example : Write statement in FOL

4. Not all students like both maths and science

$\sim \forall x (\text{students}(x) \rightarrow \text{like}(x, \text{maths}) \wedge \text{like}(x, \text{science}))$

Example : Write statement in FOL

More Complex Examples:

Multiple Quantifiers (Same Type):

Ex: Everyone respects everyone

$\forall x \forall y \text{ respects}(x, y)$

Multiple Quantifiers (Different Type):

Ex. Everyone is loyal to Someone

$\forall x \exists y \text{ loyal}(x, y)$

Example : Write statement in FOL

Mixed Quantifiers:

Ex: "Every student has at least one advisor"

$$\forall x \exists y (\text{Student}(x) \rightarrow \text{Advisor}(y, x))$$

Order matters

$$\exists y \forall x (\text{Student}(x) \rightarrow \text{Advisor}(y, x))$$

would mean one advisor for all students

Example : Write statement in FOL

Negation with Quantifiers:

"Not all birds can fly"

- $\neg \forall x (\text{Bird}(x) \rightarrow \text{CanFly}(x))$

"Everyone has a parent"

- $\forall x \exists y (\text{Person}(x) \rightarrow \text{Parent}(y, x))$

Example : Write s

All students taking AI are genius.

Some students taking AI are genius and hardworking.

Examples (from sentences to FOL)

1. All cats are cute.

- $\forall x \text{ Cat}(x) \Rightarrow \text{Cute}(x)$

1a. Cat of Noman is cute.

$\text{Cute}(\text{CatOf}(\text{Noman}))$

2. All students taking AI are genius.

- $\forall x \text{ Student}(x) \wedge \text{TakingAI}(x) \Rightarrow \text{Genius}(x)$

3. Some students taking AI are genius and hardworking.

- $\exists x \text{ Student}(x) \wedge \text{TakingAI}(x) \Rightarrow \text{Genius}(x) \wedge \text{Hardworking}(x)$

Example : Write statement in FOL

Properties of quantifiers

1. $\forall x \forall y$ is the same as $\forall y \forall x$
2. $\exists x \exists y$ is the same as $\exists y \exists x$
3. $\exists x \forall y$ is not the same as $\forall y \exists x$
 $\exists x \forall y \text{ Loves}(x,y)$
 - ✓ “There is a person who loves everyone in the world”
 $\forall y \exists x \text{ Loves}(x,y)$
 - ✓ “Everyone in the world is loved by at least one person”

Example : Wri

Translating English to FOL

Every gardener likes the sun.

$$\forall x \text{ gardener}(x) \rightarrow \text{likes}(x, \text{Sun})$$

You can fool some of the people all of the time.

$$\exists x \forall t \text{ person}(x) \wedge \text{time}(t) \rightarrow \text{can-fool}(x, t)$$

You can fool all of the people some of the time.

$$\forall x \exists t (\text{person}(x) \rightarrow \text{time}(t) \wedge \text{can-fool}(x, t))$$

$$\forall x (\text{person}(x) \rightarrow \exists t (\text{time}(t) \wedge \text{can-fool}(x, t)))$$

Equivalent

All purple mushrooms are poisonous.

$$\forall x (\text{mushroom}(x) \wedge \text{purple}(x)) \rightarrow \text{poisonous}(x)$$

No purple mushroom is poisonous.

$$\neg \exists x \text{ purple}(x) \wedge \text{mushroom}(x) \wedge \text{poisonous}(x)$$

$$\forall x (\text{mushroom}(x) \wedge \text{purple}(x)) \rightarrow \neg \text{poisonous}(x)$$

Equivalent

There are exactly two purple mushrooms.

$$\begin{aligned} \exists x \exists y \text{ mushroom}(x) \wedge \text{purple}(x) \wedge \text{mushroom}(y) \wedge \text{purple}(y) \wedge \neg(x=y) \wedge \forall z \\ (\text{mushroom}(z) \wedge \text{purple}(z)) \rightarrow ((x=z) \vee (y=z)) \end{aligned}$$

Clinton is not tall.

$$\neg \text{tall}(\text{Clinton})$$

X is above Y iff X is on directly on top of Y or there is a pile of one or more other objects directly on top of one another starting with X and ending with Y.

$$\forall x \forall y \text{ above}(x, y) \leftrightarrow (\text{on}(x, y) \vee \exists z (\text{on}(x, z) \wedge \text{above}(z, y)))$$

Example : Write state

Every person who buys a policy is smart.

There is an agent who sells policies only to people who are not insured.

e.) Sentence :
Every person who buys a policy is smart.

Translation :
 $(\forall x)(\forall y) \text{ Person}(x) \wedge \text{Policy}(y) \wedge \text{Buys}(x,y) \Rightarrow \text{Smart}(x)$

f.) Sentence :
No person buys an expensive policy.

Translation :
 $(\forall x)(\forall y) \text{ Person}(x) \wedge \text{Policy}(y) \wedge \text{Buys}(x,y) \Rightarrow \sim \text{Expensive}(y)$

g.) Sentence :
There is an agent who sells policies only to people who are not insured.

Translation :
 $(\exists x)(\text{Agent}(x) \wedge (\forall y) \text{SellsPolicy}(x,y) \Rightarrow \sim \text{Insured}(y))$

Note : Here the students might choose to use a more general predicate, $\text{Sells}(x,y,z)$, i.e x sells y to z and have a separate predicate called $\text{Policy}(y)$.

h.) Sentence :
There is a barber who shaves all men in town who do not shave themselves.

Translation :
 $(\exists x)(\text{Barber}(x) \wedge (\forall y) \text{Man}(y) \wedge \text{Shaves}(x,y) \Rightarrow \sim \text{Shaves}(y,y))$

i.) Sentence :
A person born in the UK, each of whose parents is a UK citizen or a UK resident, is a UK citizen by birth.

Translation:
 $(\forall x)((\text{Born}(x,\text{UK}) \wedge (\forall y)(\text{Parent}(y,x) \wedge (\text{Citizen}(y,\text{UK}) \vee \text{Resident}(y,\text{UK})))) \Rightarrow \text{CitizenByBirth}(x,\text{UK}))$

j.) Sentence :
A person born outside the UK, one of whose parents is a UK citizen by birth, is a UK citizen by descent.

Translation :
 $(\forall x)((\sim \text{Born}(x,\text{UK}) \wedge (\exists y)(\text{Parent}(y,x) \wedge (\text{CitizenByBirth}(y,\text{UK})))) \Rightarrow \text{CitizenByDescent}(x,\text{UK}))$

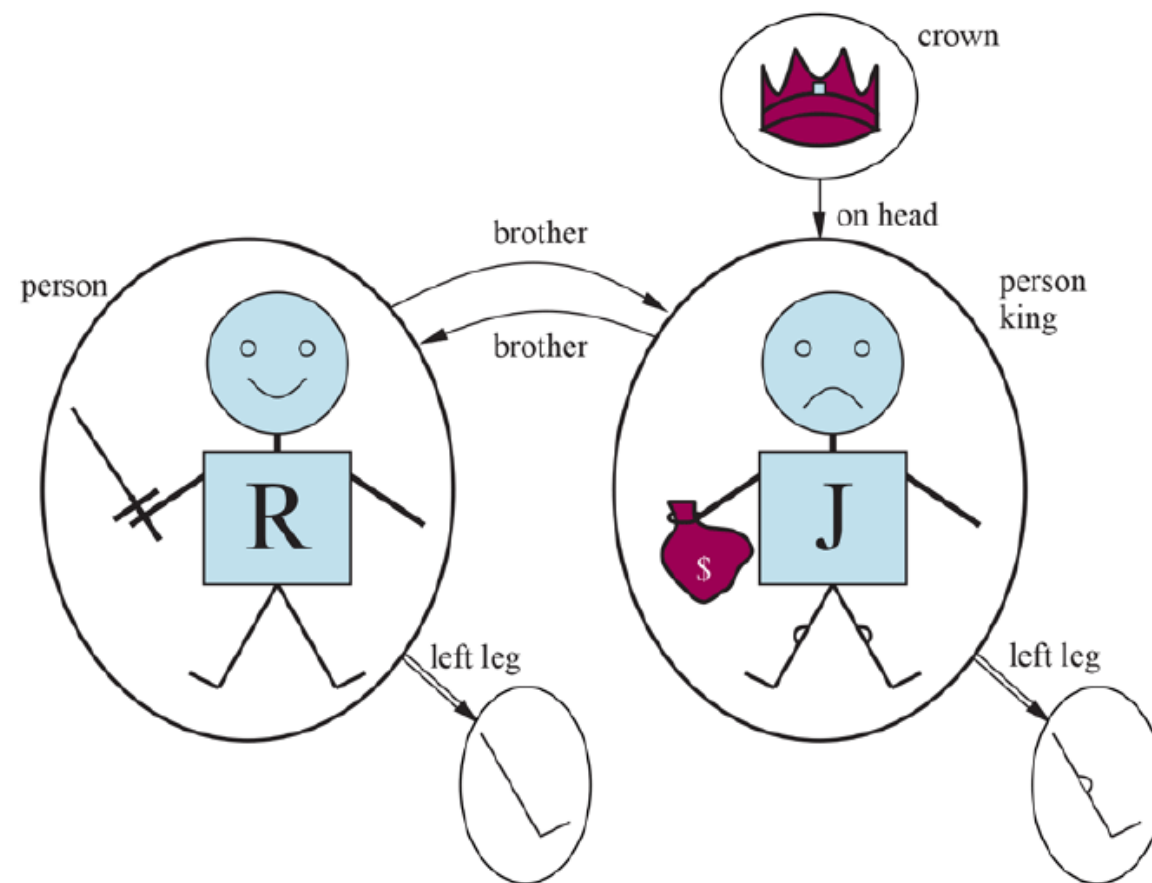
Models for first-order logic

Object

1. Richard the Lionheart, King of England from 1189 to 1199;
2. his younger brother, the evil King John, who ruled from 1199 to 1215;
3. the left leg of Richard
4. the left leg of John
5. Crown

A tuple is a collection of objects arranged in a fixed order.

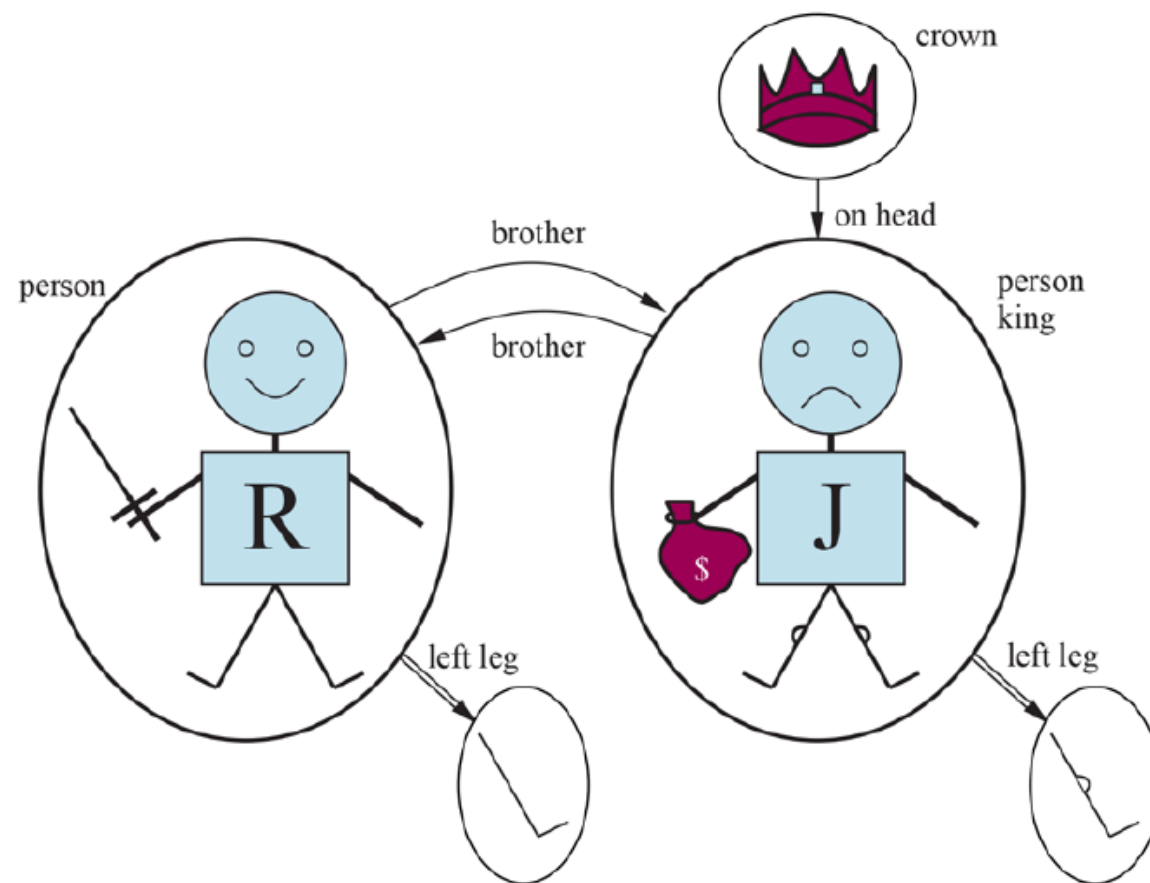
<Richard the Lionheart, King John>



A model containing five objects, two binary relations (brother and on-head), three unary relations (person, king, and crown), and one unary function (left-leg).

Models for first-order logic

- **The symbols are of three kinds:**
- Constant symbols: Stand for objects
- Predicate symbols: Stand for relations
- Function symbols: Stand for functions.
- **Example:**
- Constant symbols: Richard and John;
- Predicate symbols: Brother, OnHead, Person, King, and Crown
- Function symbol : LeftLeg.

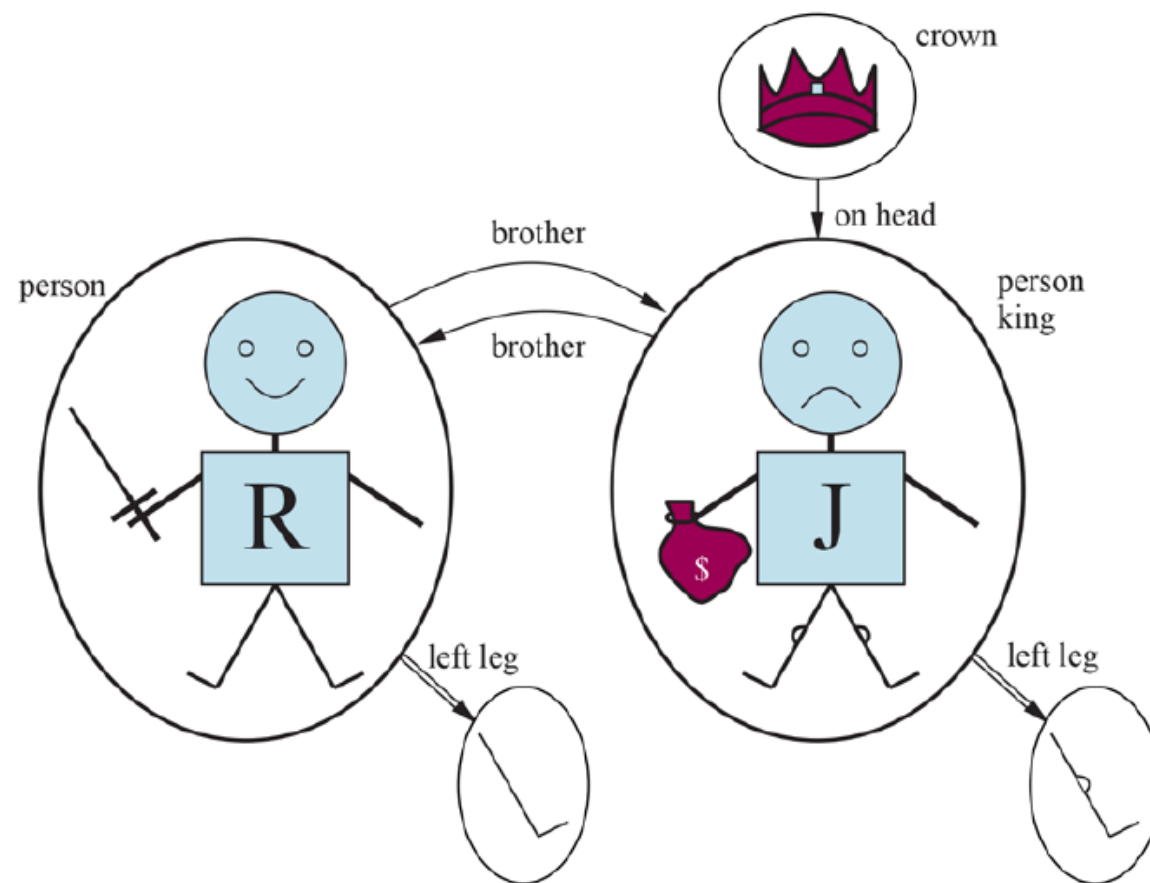


A model containing five objects, two binary relations (brother and on-head), three unary relations (person, king, and crown), and one unary function (left-leg).

Models for first-order logic

1. The objects in the model may be **related** in various ways.
2. Richard and John are brothers.
3. A **relation** is just the set of tuples of objects that are related.
4. Thus, the **brotherhood relation** in this model is the set

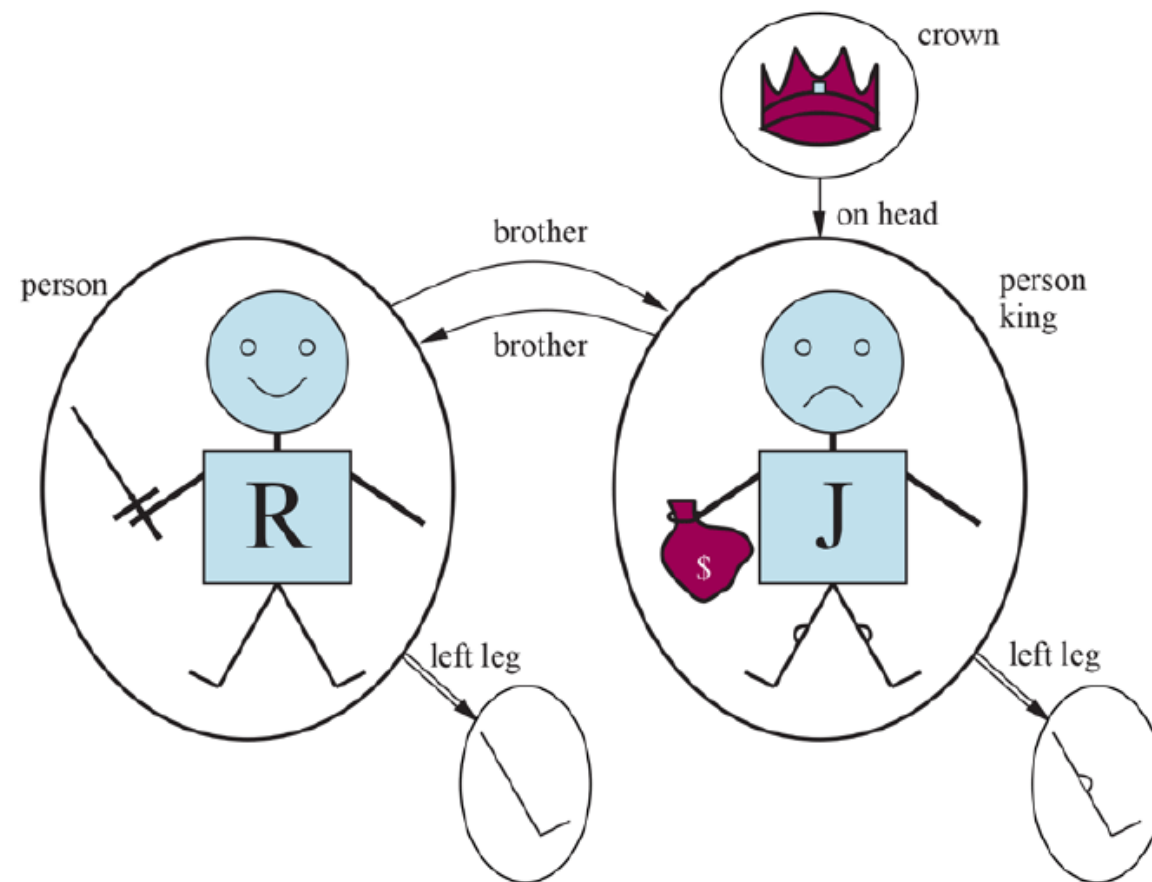
$\{\langle \text{Richard the Lionheart}, \text{King John} \rangle, \langle \text{King John}, \text{Richard the Lionheart} \rangle\}$.



A model containing five objects, two binary relations (brother and on-head), three unary relations (person, king, and crown), and one unary function (left-leg).

Models for first-order logic

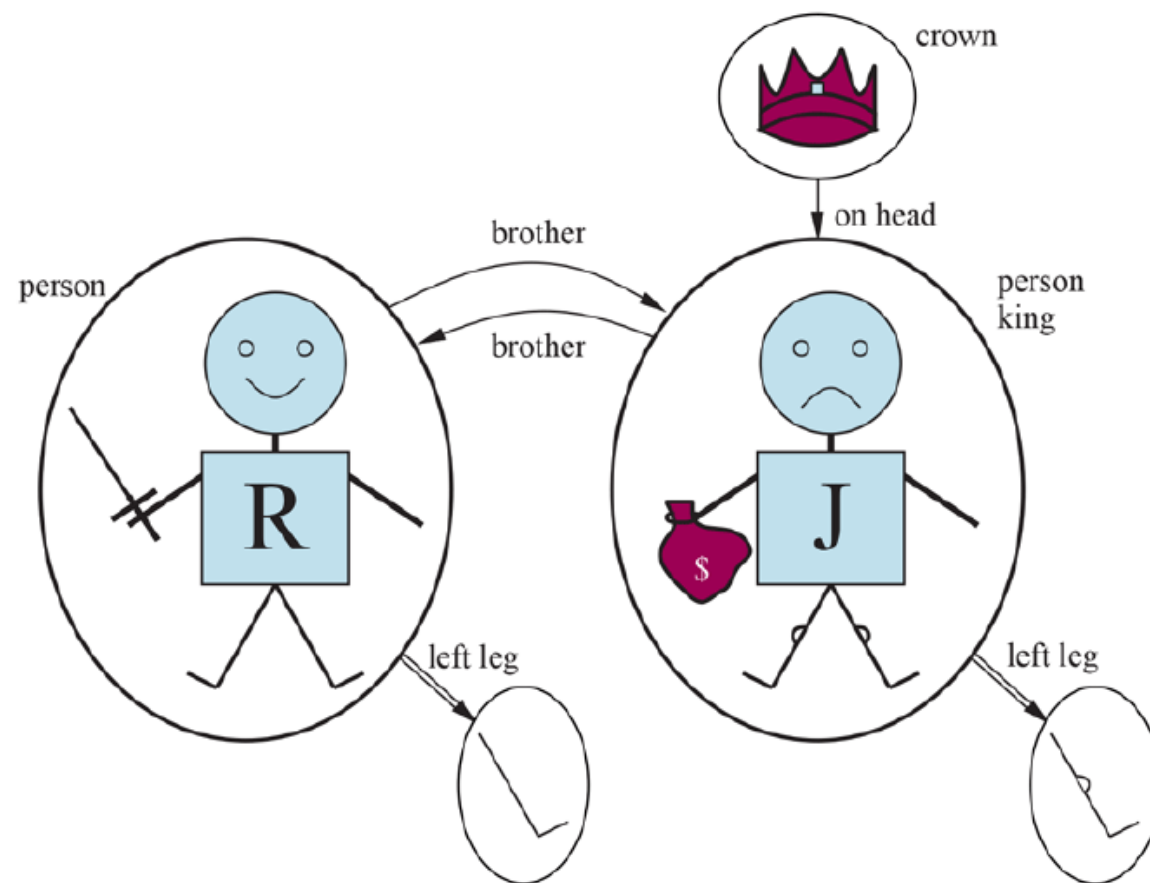
- The crown is on King John's head, so the “on head” relation contains just one tuple, $\langle \text{the crown, King John} \rangle$.
- The “brother” and “on head” relations are **binary relations**—that is, they relate pairs of objects.
- The model also contains **unary relations**, or properties:
 - the “**person**” property is true of both Richard and John;
 - the “**king**” property is true only of John (presumably because Richard is dead at this point); he



A model containing five objects, two binary relations (brother and on-head), three unary relations (person, king, and crown), and one unary function (left-leg).

Models for first-order logic

- **Functions**, in that a given object must be related to **exactly one object**.
- For example, each person has one left leg, so the model has a unary “left leg” function



A model containing five objects, two binary relations (brother and on-head), three unary relations (person, king, and crown), and one unary function (left-leg).

Using First Order logic

Assertions and queries in first-order logic

Sentences are added to a knowledge base using TELL, exactly as in propositional logic. Such sentences are called **assertions**.

For example, we can assert that

John is a king, Richard is a person, and all kings are persons

- $\text{TELL}(\text{KB}, \text{King}(\text{John}))$.
- $\text{TELL}(\text{KB}, \text{Person}(\text{Richard}))$.
- $\text{TELL}(\text{KB}, \forall x \text{ King}(x) \Rightarrow \text{Person}(x))$

Using First Order logic

Assertions and queries in first-order logic

We can ask questions of the knowledge base using ASK.

Questions asked with ASK are called **queries or goals**.

For example, **ASK(KB, King(John))**

Unary Relations:

1. **Person:** Both R and J are labeled as people.
 - $\text{Person}(R)$
 - $\text{Person}(J)$
2. **King:** J is labeled as a king.
 - $\text{King}(J)$
3. **Crown:** The object with the crown is labeled.
 - $\text{Crown}(C)$ where C denotes the crown object.

Binary Relations:

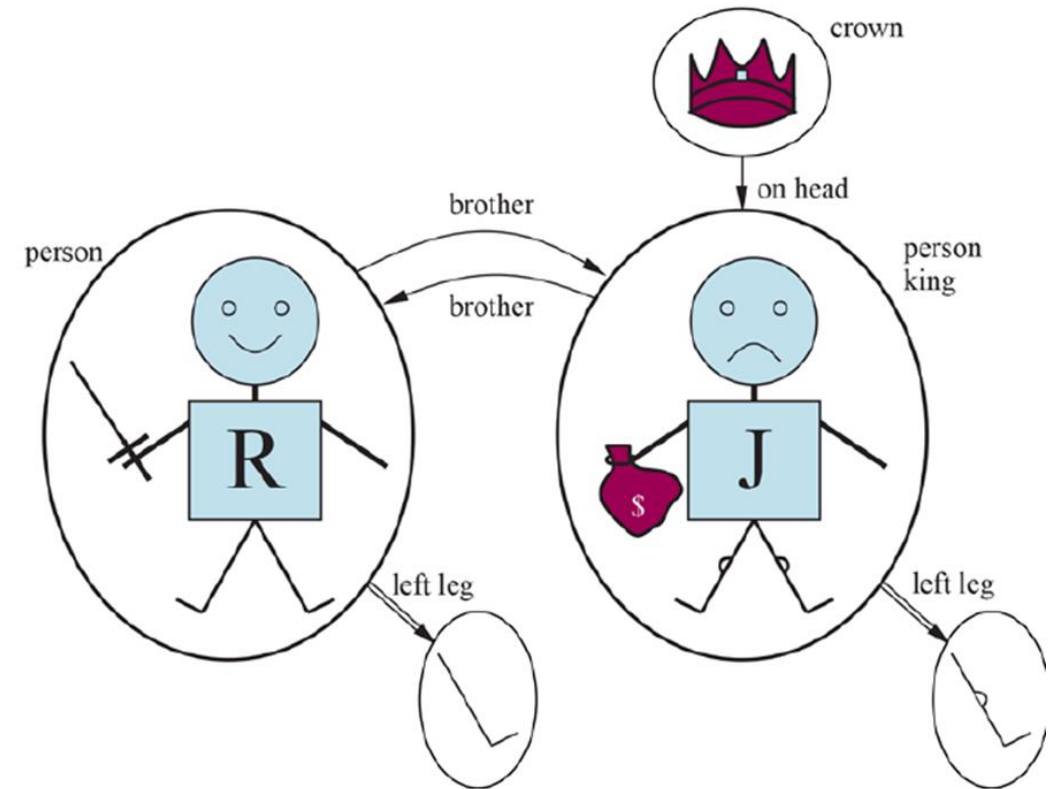
1. **Brother:** R and J are marked as brothers to each other.
 - $\text{Brother}(R, J)$
 - $\text{Brother}(J, R)$
2. **On-Head:** The crown is on the head of J.
 - $\text{OnHead}(C, J)$

Unary Function:

1. **Left-Leg:** Both R and J have left legs associated with them.
 - $\text{LeftLeg}(R) = L_1$
 - $\text{LeftLeg}(J) = L_2$



Write Statement in first-order logic



A model containing five objects, two binary relations (brother and on-head), three unary relations (person, king, and crown), and one unary function (left-leg).

Knowledge Engineering in first-order logic

- The general process of constructing knowledge base is called knowledge engineering
- A knowledge Engineer investigates a
 - **Particular** domain
 - **learns** what concepts are important in that domain
 - **creates** a formal representation of the objects and relations in the domain

Knowledge Engineering in first-order logic

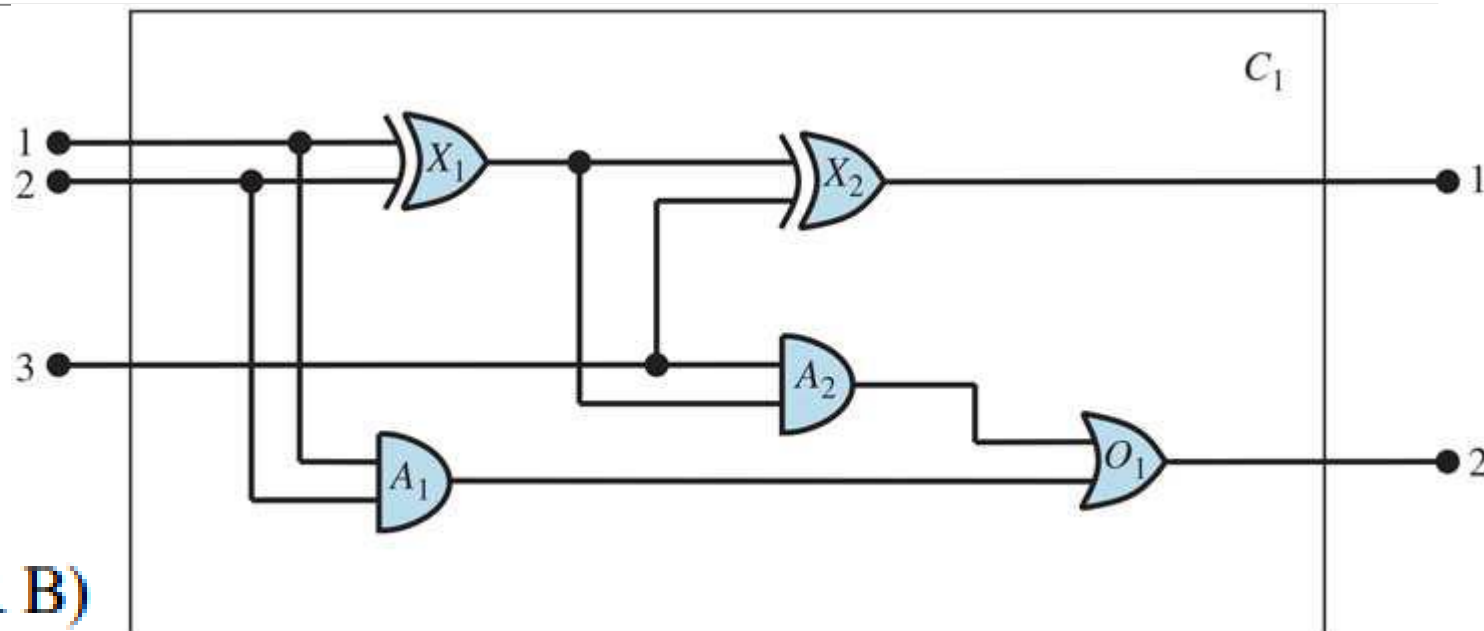
- **Steps in the knowledge-engineering process**
 1. Identify the questions
 2. Assemble the relevant knowledge
 3. Decide on a vocabulary of predicates, functions, and constants
 4. Encode general knowledge about the domain
 5. Encode a description of the specific problem instance
 6. Pose queries to the inference procedure and get answers
 7. Debug the knowledge base

The Electronic Circuits Domain: One-Bit Full Adder

Develop an ontology and knowledge base that allow us to reason about digital circuits

$$\text{SUM} = \text{C-IN XOR (A XOR B)}$$

$$\text{COÛT} = \text{AB} + \text{C-IN (A EX - OR B)}$$



A digital circuit C_1 , is a one-bit full adder. The first two inputs are the two bits to be added, and the third input is a carry bit. The first output is the sum, and the second output is a carry bit for the next adder. The circuit contains two XOR gates, two AND gates, and one OR gate.

Identify

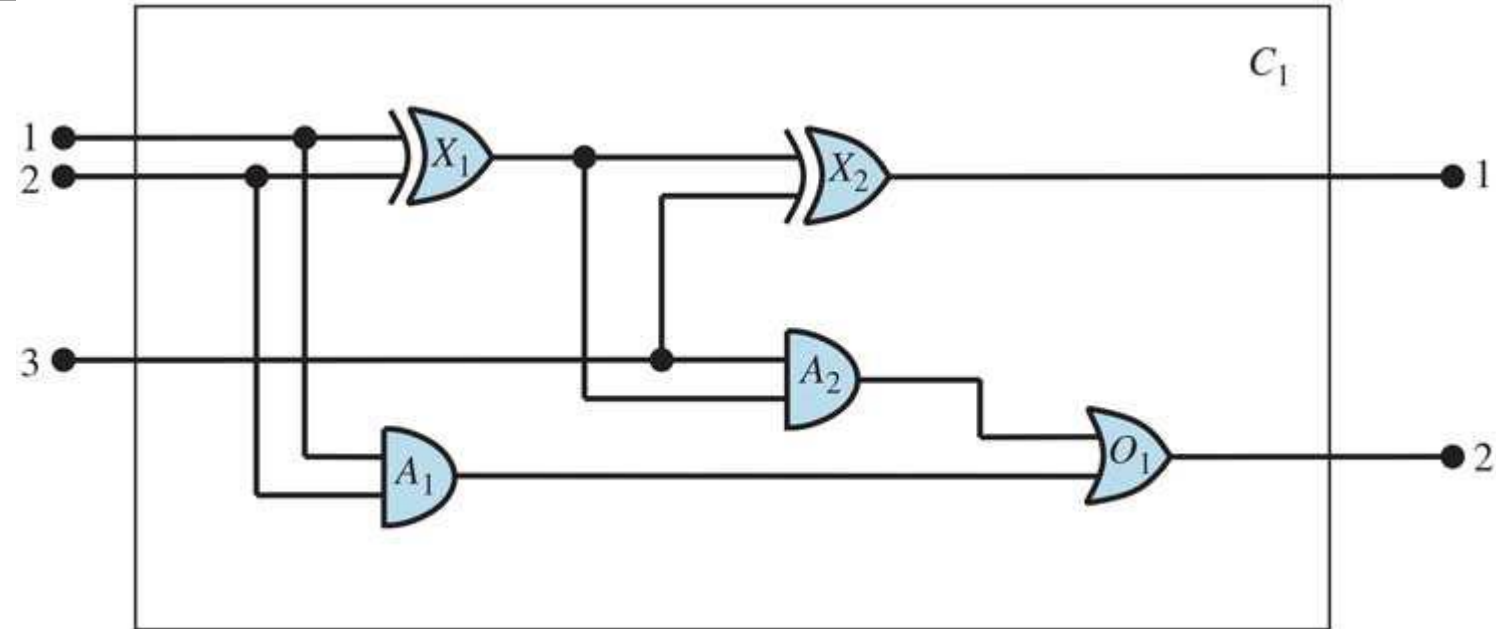
The Electronic Circuits Domain

Applications of using Knowledge Base for Digital Circuits:

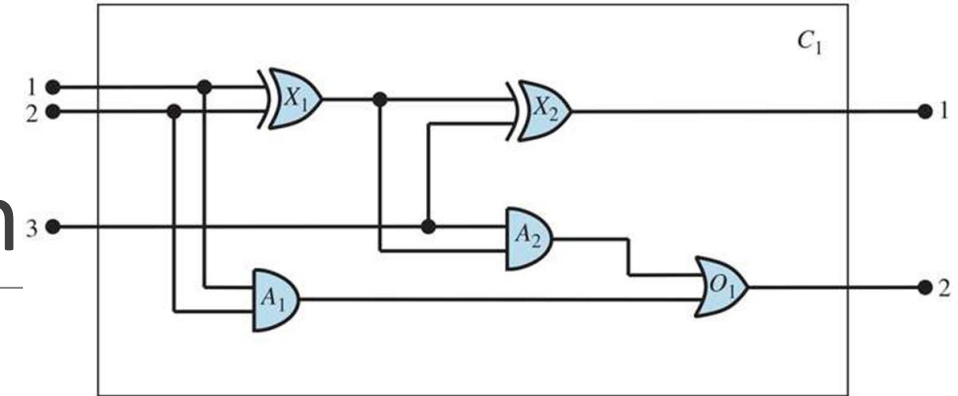
1. Automated Circuit Simulation
2. Fault Detection and Diagnosis
3. Circuit Design Verification and Validation
4. Optimization of Circuit Layout
5. Configuration and Reconfiguration of Digital Circuits

Tools and Technology:

OWL (Web Ontology Language) for defining the ontology and relationships of digital components.



The Electronic Circuits Domain



Steps in the knowledge-engineering process

1. Identify the task

At the first level, we will examine the functionality of the circuit:

Does the circuit add properly?

What will be the output of gate A2, if all the inputs are high?

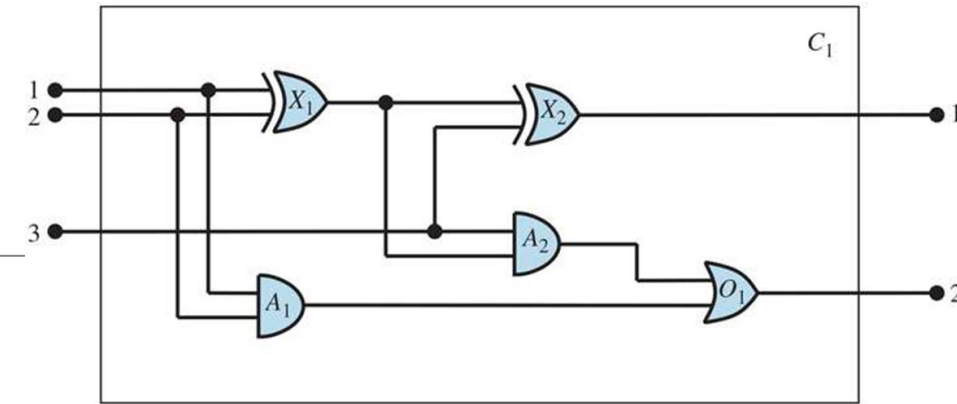
At the second level, we will examine the circuit structure details such as:

Which gate is connected to the first input terminal?

Does the circuit have feedback loops?

The Electronic Circuits Domain

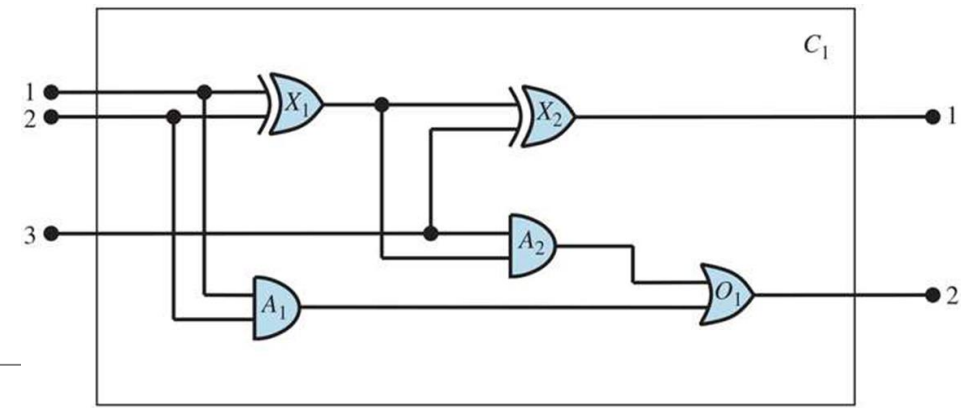
2. Assemble the relevant knowledge



For digital circuits, the following **required knowledge**:

- Logic circuits are made up of wires and gates. Signal flows through wires to the input terminal of the gate, and each gate produces the corresponding output which flows further.
- In this logic circuit, there are four types of gates used: AND, OR, XOR, and NOT.
- **Collect Domain Knowledge:** Gather information from textbooks, domain experts, and digital circuit manuals. This includes the behavior of different digital components, logical rules, and constraints specific to digital circuit design.
- **Establish Relationships:** Identify the relationships between components (e.g., inputs to outputs, signal flow, timing dependencies)

The Electronic Circuits Domain

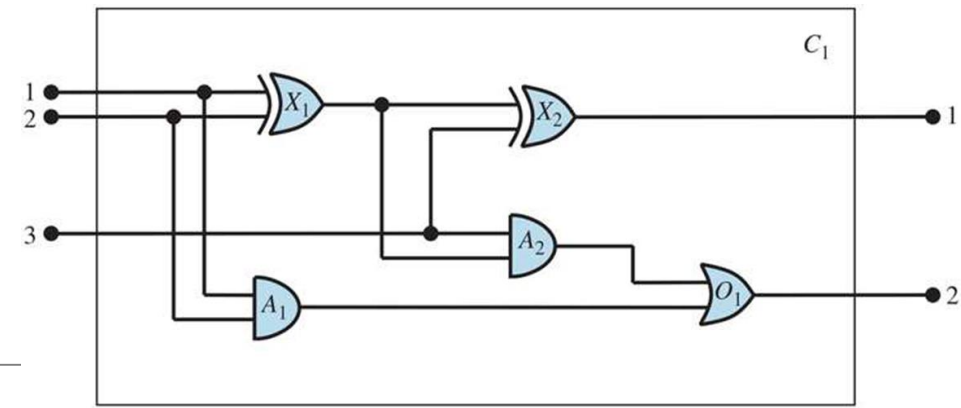


3. Decide on a vocabulary of predicates, functions, and constants

- Select functions, predicate, and constants to represent the circuits, terminals, signals, and gates.
- Each gate is represented as an **object** which is named by a constant, **X1**
- Circuits will be identified by a **predicate**: **Circuit (C1)**
-
- **Objects have two signal values are 1 and 0, and a function Signal(t)** denotes the signal value for the terminal t.
- The functionality of each gate is determined by its type, which is taken as constants such as **AND, OR, XOR, or NOT**.
- For **gate input function**: **In(1, X1)** - first input terminal of the gate x1
 - **Out (1, X1)** - output terminal

The connectivity between gates can be represented by predicate **Connect(Out(1, X1), In(2, A2))**.

The Electronic Circuits Domain



3. Decide on a vocabulary of predicates, functions, and constants

$Terminal(t_1)$

$Out(1,g)$

$Connected(t_1, t_2)$

$Type(g) \Rightarrow k$

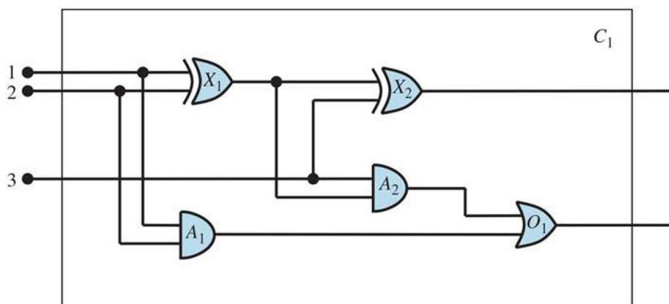
$Signal(t) = 0.$

$Gate(g)$

$In(2,g)$

4. Encode general knowledge about the domain

To encode the general knowledge about the logic circuit, we need some following rules:



1. If two terminals are connected, then they have the same signal:

$$\forall t_1, t_2 \text{Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2) .$$

2. The signal at every terminal is either 1 or 0:

$$\forall t \text{Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0.$$

3. Connected is commutative:

$$\forall t_1, t_2 \text{Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1).$$

4. There are four types of gates:

$$\forall g \text{Gate}(g) \wedge k = \text{Type}(g) \Rightarrow k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR} \vee k = \text{NOT}.$$

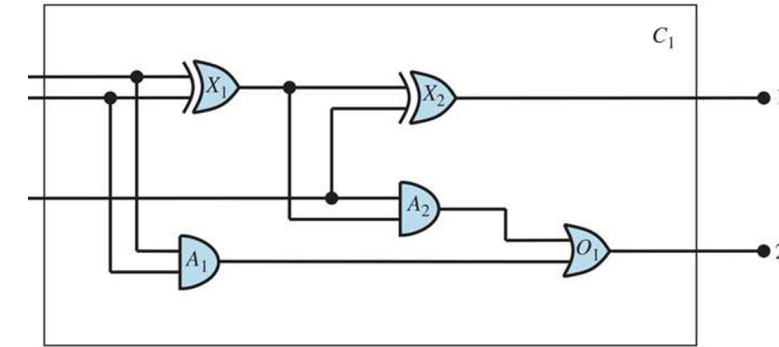
The Electronic Circuits Domain

5. An AND gate's output is 0 if and only if any of its inputs is 0:

$$\forall_g \text{Gate}(g) \wedge \text{Type}(g) = \text{AND} \Rightarrow \text{Signal}(\text{Out}(1,g)) = 0 \Leftrightarrow \exists n \text{Signal}(\text{In}(n,g)) = 0 .$$

6. An OR gate's output is 1 if and only if any of its inputs is 1:

$$\forall_g \text{Gate}(g) \wedge \text{Type}(g) = \text{OR} \Rightarrow \text{Signal}(\text{Out}(1,g)) = 1 \Leftrightarrow \exists n \text{Signal}(\text{In}(n,g)) = 1 .$$



The Electronic Circuits Domain

7. An XOR gate's output is 1 if and only if its inputs are different:

$$\forall_g \text{Gate}(g) \wedge \text{Type}(g) = \text{XOR} \Rightarrow \text{Signal}(\text{Out}(1,g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(n,g)) \neq \text{Signal}(\text{In}(2,g)) .$$

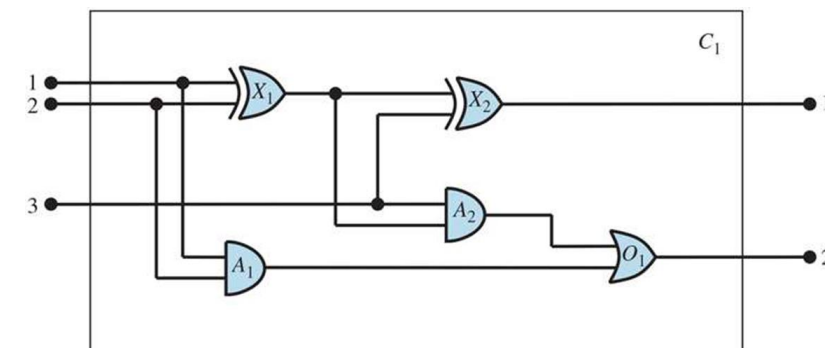
8. A NOT gate's output is different from its input:

$$\forall_g \text{Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow \text{Signal}(\text{Out}(1,g)) \neq \text{Signal}(\text{In}(1,g)) .$$

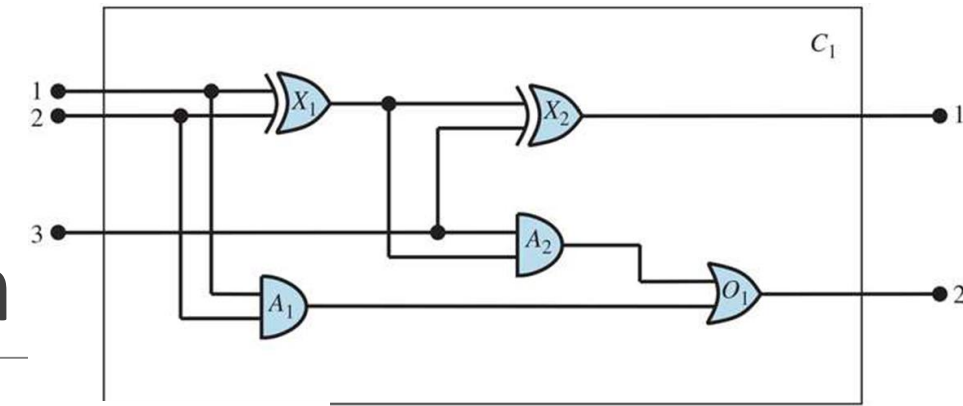
9. The gates (except for NOT) have two inputs and one output.

$$\forall_g \quad \text{Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow \text{Arity}(g,1,1).$$

$$\forall_g \quad \text{Gate}(g) \wedge k = \text{Type}(g) \wedge (k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR}) \Rightarrow \text{Arity}(g,2,1).$$



The Electronic Circuits Domain



10. A circuit has terminals, up to its input and output arity, and nothing beyond its arity:

$$\begin{aligned} \forall c, i, j \text{ Circuit}(c) \wedge \text{Arity}(c, i, j) &\Rightarrow \\ \forall n \ (n \leq i \Rightarrow \text{Terminal}(\text{In}(n, c))) \wedge (n > i \Rightarrow \text{In}(n, c) = \text{Nothing}) \wedge \\ \forall n \ (n \leq j \Rightarrow \text{Terminal}(\text{Out}(n, c))) \wedge (n > j \Rightarrow \text{Out}(n, c) = \text{Nothing}) \end{aligned}$$

11. Gates, terminals, and signals are all distinct.

$$\forall g, t, s \text{ Gate}(g) \wedge \text{Terminal}(t) \wedge \text{Signal}(s) \Rightarrow g \neq t \wedge g \neq s \wedge t \neq s.$$

12. Gates are circuits.

$$\forall g \text{ Gate}(g) \Rightarrow \text{Circuit}(g)$$

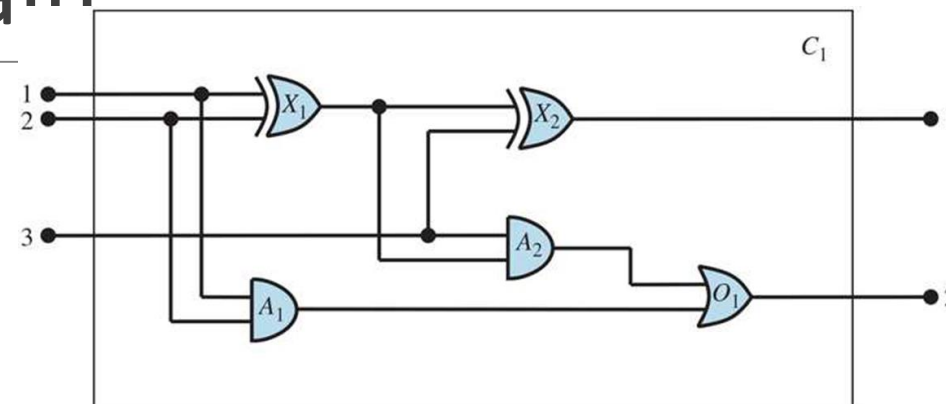
The Electronic Circuits Domain

5. Encode a description of the specific problem instance

This step write simple atomic sentences of instances of concepts

For the given circuit C1, we can encode the problem instance in atomic sentences :

In circuit there are two XOR, two AND, and one OR gate so atomic sentences for these gates will be:



$Circuit(C_1) \wedge Arity(C_1, 3, 2)$

$Gate(X_1) \wedge Type(X_1) = XOR$

$Gate(X_2) \wedge Type(X_2) = XOR$

$Gate(A_1) \wedge Type(A_1) = AND$

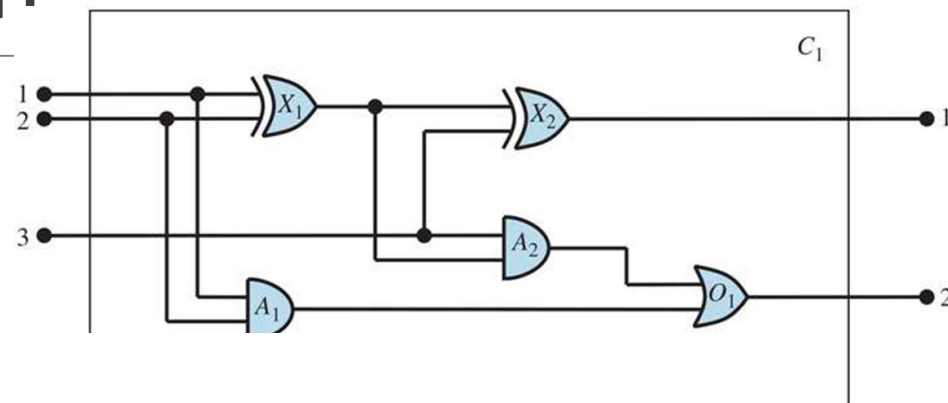
$Gate(A_2) \wedge Type(A_2) = AND$

$Gate(O_1) \wedge Type(O_1) = OR.$

The Electronic Circuits Domain

5. Encode a description of the specific problem instance

Then we show the connections between them:



Connected(Out(1,X₁),In(1,X₂))

Connected(Out(1,X₁),In(2,A₂))

Connected(Out(1,A₂),In(1,O₁))

Connected(Out(1,A₁),In(2,O₁))

Connected(Out(1,X₂),Out(1,C₁))

Connected(Out(1,O₁),Out(2,C₁))

Connected(In(1,C₁),In(1,X₁))

Connected(In(1,C₁),In(1,A₁))

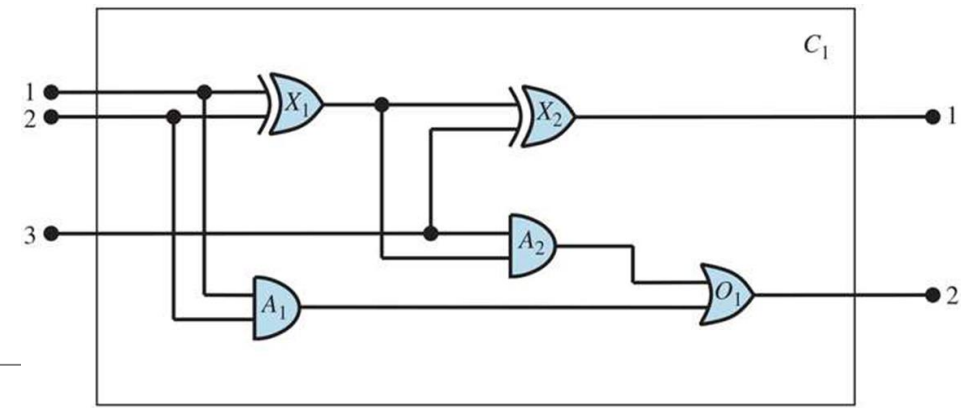
Connected(In(2,C₁),In(2,X₁))

Connected(In(2,C₁),In(2,A₁))

Connected(In(3,C₁),In(2,X₂))

Connected(In(3,C₁),In(1,A₂)) .

The Electronic Circuits Domain



6. Pose queries to the inference procedure and get answers

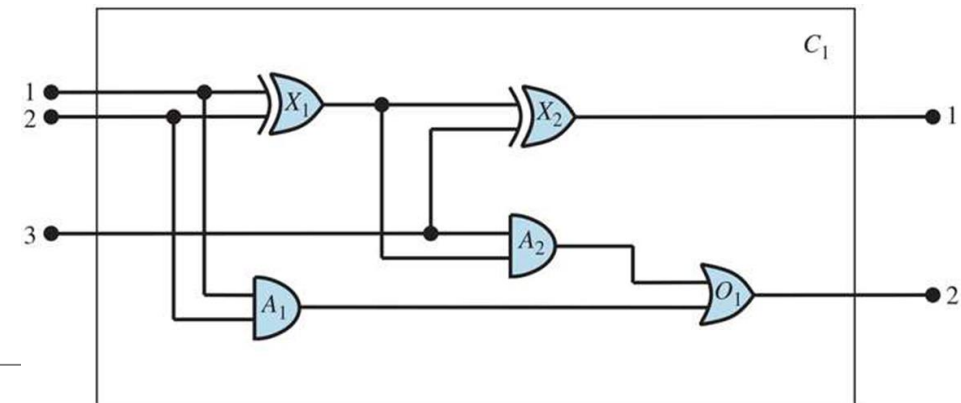
Find all the possible set of values of all the terminal for the adder circuit.

The first query will be: What should be the combination of input which would generate the first output of circuit C1 as 0 and a second output to be 1?

$$\begin{aligned} &\exists i1, i2, i3 \text{ Signal (In(1, C1))=i1 } \wedge \text{Signal (In(2, C1))=i2 } \wedge \text{Signal (In(3, C1))= i3} \\ &\wedge \text{Signal (Out(1, C1)) =0 } \wedge \text{Signal (Out(2, C1))=1} \end{aligned}$$

This final query will return a complete input–output table for the device, which can be used to check that it does in fact add its inputs correctly. This is a simple example of circuit **verification**.

The Electronic Circuits Domain



7. Debug the knowledge base

In this step, we will try to debug the issues of knowledge base.

$\text{Signal}(\text{Out}(1, X1)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, X1)) \neq \text{Signal}(\text{In}(2, X1))$.

$\text{Signal}(\text{Out}(1, X1)) = 1 \Leftrightarrow 1 \neq 0$.